

[← Go Back](#)

Hardware

Edge Control

Tutorials

[Edge Control User Manual](#)[Smart Farm Irrigation
System Using Arduino®
Edge Control](#)[LPWAN Irrigation System
Using Arduino® Edge
Control](#)[Connecting and Controlling
a Motorized Ball Valve](#)[Home](#) / [Hardware](#) / [Edge Control](#) /[LPWAN Irrigation System Using Arduino®
Edge Control](#)

LPWAN Irrigation System Using Arduino® Edge Control

This application note describes how to control a four zones irrigation system using Edge Control and Arduino Cloud with LoRa® connectivity.

Author · Christopher Mendez · Last revision · 02/28/2025

ON THIS PAGE

Introduction

[Goals](#)[Hardware and Software
Requirements](#) +[Irrigation System Setup](#)[Irrigation System Overview](#) +[Connectivity](#) +[Full Smart Irrigation System
Example](#) +[Conclusion](#) +[Trademark Acknowledgments](#)

Introduction

Agricultural & farm activities are normally carried out in remote environments, where having access to consistent electricity power sources and good communication connectivity is a challenge.

Smart farming techniques are being implemented more and more due to the importance of optimizing the use of resources while increasing the demand for more efficient, eco-friendly, and profitable crops.



Application Note Overview.
Each pot represents one
individual irrigation zone
capable of watering a crop

field

[Help](#)

Implementing traditional wired communication infrastructure in remote areas can be expensive and time-consuming. LoRa® technology, being a wireless technology, provides a cost-effective alternative, as it requires minimal infrastructure setup, reducing installation and maintenance costs.

Arduino has you covered in these scenarios with its Pro solutions, including products designed to work in remote environments, supplying their power from renewable sources, and providing long-distance connectivity and low power consumption.

Goals

The goal of this application note is to showcase a LoRa® technology based farming irrigation system that can be implemented on real agriculture fields using a combination of an Edge Control, an MKR WAN 1310, and the Arduino Cloud. The project's objectives are the following:

- Independently control four irrigation zones using latching valves.

- Leverage MKR WAN 1310 with LoRa® technology and a Wisgate (Lite or PRO) to communicate with Arduino Cloud.

- Monitor soil moisture and decide whether to irrigate based on it.

- Display the soil humidity level on the Edge Control Enclosure kit LCD.

- Manually activate irrigation through Enclosure Kit built-in push button.

- Monitor average humidity level, irrigation time and water consumption on dedicated charts on Arduino Cloud.

evaluate the amount of consumed water.

Hardware and Software Requirements



Project main hardware

Hardware Requirements

Arduino Edge Control (x1)

Arduino MKR WAN 1310 (x1)

Arduino Edge Control Enclosure Kit (x1)

Water flow sensor (YF-B2 DN15)
(x1)

Watermark Soil Moisture Sensors (200SS) (x4)

2-Wires Latching Solenoid Valves
(Galcon YLZ S1602) (x4)

12 VDC 5Ah acid/lead SLA battery
(NP12-7Ah) (x1)

18 VDC 180 W solar panel
(BougeRV 180W 9BB)(x1)

3.4 meters of DN15 PVC pipes
(x1)

DN15 PVC TEE pipes (x3)

DN15 PVC elbow (x8)

DN15 Manual Valve (x1)

DN15 PVC caps (x4)

DN15 PVC male adapters (x11)

DN15 wall pipe brackets (x7)

DN15 to DN20 adapters (x8)

Rectangular planters (x4)

6 meters of duplex cable AWG 18 (x1)

Electrical Register Box (x1)

Access to the water system, in case you do not have access to the water system and you want to use a water tank instead, please take also a look at the [Smart Farm Irrigation](#) application note to know more.

Software Requirements

[Arduino IDE 1.8.10+](#), [Arduino IDE 2](#), or [Arduino Cloud Editor](#).

If you are going to use an offline Arduino IDE, you must install the following libraries:

```
Arduino_EdgeControl ,  
RunningMedian ,  
ArduinoIoTCloud ,  
ArduinoJson and  
Arduino_ConnectionsHandler .
```

You can install them through the Arduino IDE Library Manager.

The [Irrigation System Arduino Sketches](#).

[Arduino Create Agent](#) to provision the MKR WAN 1310 on the Arduino Cloud.

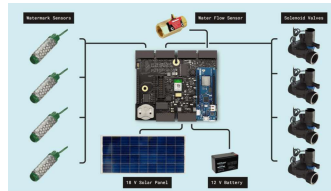
Irrigation System Setup

The Edge Control Enclosure Kit is perfect to enclose the main board alongside the MKR shields:



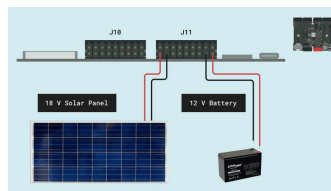
The electrical connections of the

diagram below:



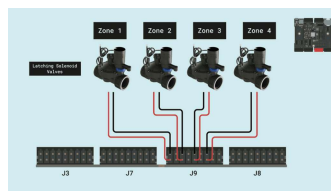
Electrical diagram of the irrigation system

The Edge Control board will be powered with a 12 VDC acid/lead SLA battery connected to BATT+ and GND of J11 respectively. The battery will be recharged with an 18 VDC 180W solar panel connected to SOLAR+ and GND on the same connector.



Power connection diagram

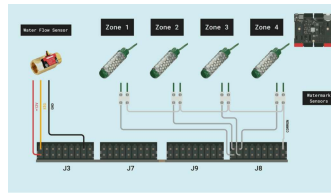
The four solenoid valves will be connected to the Edge Control latching outputs of the J9 connector following the wiring below.



Solenoid valves connection diagram

i You can also use 3-Wires motorized valves. See this [guide](#) for reference.

The water flow sensor will be connected to a +12 VDC output,



Watermark and water flow sensors connection diagram

The four watermark sensors will be connected to a terminal block rail; one terminal to the common and the others to the watermark sensor inputs from 1 to 4 respectively on J8. Depending on the wiring of your sensors, you can use electrical clamps to ease the connection.

The final wiring should be similar to the following picture:



Edge Control with all the wiring

Irrigation System Overview

The irrigation system works as a whole: it integrates the water flow measurement and the activation of the valves, done by the Edge Control, with the Cloud communication using the MKR WAN 1310.

The Edge Control is responsible for:

- Measuring the water usage with a water flow sensor.

- Measuring the soil humidity level

Controlling the LCD screen of the Edge Control Enclosure kit, where different system variables will be shown, including soil humidity.

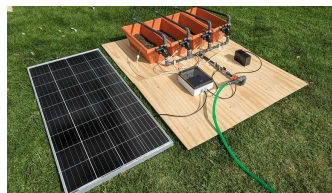
Deciding whether to irrigate based on soil local humidity.

The MKR WAN 1310 is responsible for:

Providing Cloud connectivity using LoRa® technology.

Reporting the values of the Edge Control sensors on the Cloud.

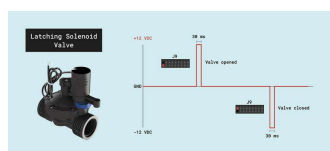
The communication between both devices is done leveraging the I2C communication protocol.



Smart irrigation system with Edge Control

Valves Control

The valves can be controlled manually by using the onboard button, one tap opens the first valve, two taps the second valve, and so on. In addition, the valves can be controlled automatically by the system when the soil moisture is poor. To do so, a "Smart mode" has to be enabled by tapping the button five times. The working time of the valves is monitored and reported on the Cloud to enable an efficient visualization of the average daily use.

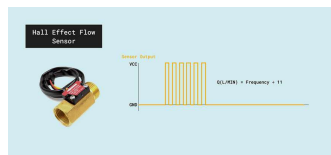




The used valves used are latching, which means they are activated by a single pulse. The polarity defines if it opens or closes. This pulse must be in the range of 20-40 ms, more than that time could damage the latching outputs of your device.

Water Usage

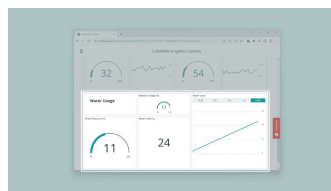
The water flow sensor will measure the consumed water and will calculate its volume in liters. This information will be monitored through the Cloud and the integrated LCD.



Water flow sensor output signal

The water flow is measured by a Hall effect sensor that generates a pulsed output by the rotation steps of a propeller inside the metal body.

The data is shown on dedicated widgets in the Arduino Cloud.



Water usage widgets in the Arduino Cloud

Soil Moisture Measurement

Instead of measuring the percentage

sensors that are capable of measuring the physical force holding the water in the soil. Those measurements are correlated with the effort plants have to make to extract water from the soil, a really interesting data for agricultural applications.

The measurement is done in Centibars, and we can use the following readings as a general guideline:

0-10 Centibars: Saturated soil

10-30 Centibars: Soil is adequately wet (except coarse sands, which are drying)

30-60 Centibars: Usual range for irrigation (most soils)

60-100 Centibars: Usual range for irrigation in heavy clay

100-200 Centibars: Soil is becoming dangerously dry - Proceed with caution!

Arduino Edge Control Code

Let's go through some important code sections to make this application fully operative; starting with the required libraries:

```

1  #include <Arduino_EdgeControl.h>
2  #include <Wire.h>
3  #include <RunningMedian.h>
4
5  #include "SensorValues.hpp"
6  #include "Helpers.h"
7
8  // The MKR1 board I2C address
9  #define EDGE_I2C_ADDR 0x60
10
11 constexpr unsigned int address = 0x60;
12
13 mbed::LowPowerTimeout timeout(10000);
14
15 // Watermark sensors threshold
16 const long open_resistance = 10000;
17
18 // Watermark sensors channels
19 uint8_t watermarkChannels[4];
20
21 constexpr float tauRatio{0.1};
22 constexpr float tauRatioS{0.1};
23 constexpr unsigned long sensorPeriod{1000};
24
25 constexpr unsigned int measurePeriod{1000};
26 RunningMedian measures{measurePeriod};
27
28 constexpr unsigned int calibrationPeriod{1000};

```

`Arduino_EdgeControl.h` will enable the support for the Edge Control peripherals; install it by searching for it on the Library Manager.

`Wire.h` will enable the I2C communication between the Edge Control, the MKR WAN 1310 and the other peripherals. It is included in the Board Support Package (BSP) of the Edge Control.

`RunningMedian.h` handles the calculations regarding the watermark sensor measurements.

There are two headers included in the project code able to handle some helper functions and structures:

`SensorValues.hpp` handles the shared variables between the

`Helpers.h` handles the real-time clock (RTC) functions to retrieve the local date and time.

This code's section also contains all the system variables regarding the following:

Constants and variables to set and store the watermark sensors data.

Constants and variables for the water flow sensor.

Flow control variables.

Structure to handle the number of button taps to control each valve manually.

```

1  /**
2   Main section setup
3  */
4  void setup() {
5
6   EdgeControl.begin();
7   Wire.begin();
8
9   delay(500);
10  Serial.begin(115200);
11  delay(2000);
12
13  Power.enable3V3();
14  Power.enable5V();
15  Power.on(PWR_3V3);
16  Power.on(PWR_VBAT);
17  Power.on(PWR_MKR1);
18
19  delay(5000); // giving
20
21  // Init Edge Control IO
22  Serial.print("IO Expand
23  if (!Expander.begin())
24    Serial.println("faile
25    Serial.println("Pleas
26    Serial.println("via P
27  } else Serial.println("
28

```

To save energy and resources, the Edge Control has different power lines that

this case, the 3.3 V, 5 V, Battery, and the MKR1 slot need to be enabled.

An external interruption is being used for the water flow sensor and the LCD button, they are attached to the IRQ_CH1 and the POWER_ON inputs respectively. To handle all the I/Os, the I/O Expander together with the Enclosure Kit LCD and the sensors inputs need to be initialized.

The loop function is in charge of handling the system's repetitive tasks.

```
1 void loop() {  
2  
3   // LCD button taps dete  
4   detectTaps();  
5   tapsHandler();  
6  
7   // reset the valves acc  
8   if (getLocalhour() == "  
9     Serial.println("Reset  
10    vals.z1_on_time_local  
11    vals.z2_on_time_local  
12    vals.z3_on_time_local  
13    vals.z4_on_time_local  
14    delay(1000);  
15  }  
16  
17  readWatermark();  
18  
19  if ((millis() - oldTime  
20  {  
21    oldTime2 = millis();  
22    readWaterFlow();  
23    auto vbat = Power.get  
24    Serial.print("Battery  
25    Serial.println(vbat);  
26    vals.battery_volt_loc  
27  }  
28
```

The Edge Control will check the number of button taps for the valve's manual control and handle the right action to do through the use of a switch case statement. Then, it will

periodically measure the battery voltage.

Every 3 minutes, the Edge Control will request the MKR WAN to send a LoRa®-based network message updating the sensors values in the Cloud.

Finally, in the loop function, we will check the valves states to control them and keep track of their active time.

Arduino MKR WAN 1310 Code

The MKR WAN 1310 needs the following libraries:

`ArduinoJson.h` parse and create JSON structures to visualize the I2C sent variables. It can be installed directly from the Arduino Library Manager.

`Wire.h` will enable the I2C communication between the Edge Control, the MKR WAN 1310 and the other peripherals. It is included in the BSP of the MKR WAN board.

There are three headers included in the project code that handles some helper functions and structures:

`thingProperties.h` is automatically generated by the Arduino Cloud. However, if you are using an offline IDE, verify it is in the same directory as your sketch and includes all the Arduino Cloud variables.

`SensorValues.hpp` handles the shared variables between the Edge Control and the MKR WAN 1310 through I2C.

`arduino_secrets.h` includes the LoRa®-based network credentials of your device.

```
1 #include "arduino_secrets.  
2 #include "thingProperties.  
3 #include "SensorValues.hpp  
4 #include <Wire.h>  
5  
6 #include <ArduinoJson.h>  
7  
8 // The MKR1 board I2C addr  
9 #define SELF_I2C_ADDR 0x05  
10  
11 unsigned long previousMill  
12 const long interval = 3*60
```

We also define the I2C address of the MKR and the update interval for the LoRa®-based network messages. Due to the LoRa technology limitations, we shouldn't define "short" intervals.

```
1 /**  
2  Main section setup  
3  */  
4 void setup() {  
5  // Initialize serial an  
6  Serial.begin(115200);  
7  // This delay gives the  
8  delay(1500);  
9  
10 // Defined in thingProp  
11 initProperties();  
12  
13 // Connect to Arduino C  
14 ArduinoCloud.begin(Ardu  
15  
16 /*  
17  The following functi  
18  related to the state  
19  the higher number th  
20  The default is 0 (on  
21  Maximum is 4  
22  */  
23  setDebugMessageLevel(2)  
24  ArduinoCloud.printDebug  
25  
26  // Init I2C coomunica  
27  Wire.begin(SELF_I2C_ADD  
28  Wire.onReceive(receiveE
```

Cloud variables, and the connection handler. Also, we initialized the I2C communication and defined a callback function `receiveEvent` to handle the received messages.

```
1  /**
2   * Function that updates t
3   * @param vals includes th
4   */
5  void uploadValues(SensorV
6
7   StaticJsonDocument<200>
8   doc["Zone 1"] = vals->v
9   doc["Zone 2"] = vals->v
10  doc["Zone 3"] = vals->v
11  doc["Zone 4"] = vals->v
12  doc["Moisture 1"] = val
13  doc["Moisture 2"] = val
14  doc["Moisture 3"] = val
15  doc["Moisture 4"] = val
16  doc["Water usage"] = va
17
18  String output;
19  serializeJson(doc, outp
20  Serial.println(output);
21
22  // Cloud variable --- S
23  water_usage = vals->wat
24  water_flow = vals->wate
25
26  z1_on_time = vals->z1_o
27  z2_on_time = vals->z2_o
28  z3_on_time = vals->z3_o
```

The `uploadValues` function simply updates the Cloud variables with the ones received from the Edge Control.

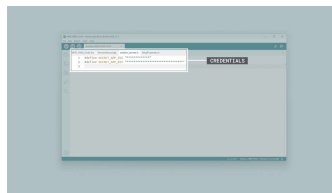
Connectivity



This project is using LoRa® technology, a long range wide area network protocol and it is a low-power wireless communication protocol designed for connecting battery-operated devices to the internet over long distances. If you want to learn more about LoRa technology, please check this [guide](#).

The MKR WAN 1310 will be the **end-device** encharged of connecting to The Things Network (TTN), which is the network server supported by the Arduino Cloud. Learn how to connect the MKR WAN 1310 to TTN using this [guide](#).

After following the guide you will get two important keys that will be needed for the LoRa® connectivity `APP_EUI` and `APP_KEY`, define them in the `arduino_secrets.h` header of your MKR code.



LoRa®-based network
credentials

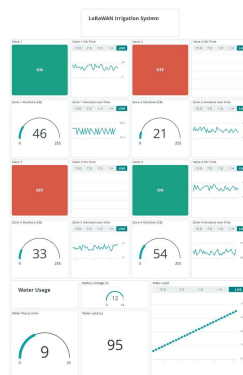
As a **gateway** we will be using the [WisGate Edge Lite 2](#), which will provide long-range coverage and access to the network. Learn how to set up yours using this [guide](#).



If there is coverage of a TTN public gateway in your area, it is not necessary to install yours. You can check your area network [here](#).

The Arduino Cloud

Taking advantage of the Arduino Cloud, it is possible to seamlessly integrate a simple but powerful dashboard to monitor and visualize the status of the system from remote, resulting in a professional Human-Computer Interaction (HCI) as shown below:



Arduino Cloud project dashboard

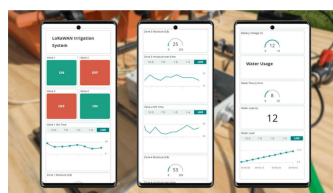
Within the Arduino Cloud's dashboard, the system variables can be monitored as follow:

Each solenoid valve status is shown as **OFF** or **ON**.

The activated time of each valve is graphed next to its state.

The watermark sensor instant values are shown in gauges alongside a time series record chart.

There is a resources section showing the battery voltage in a gauge and the water flow in liters per minute, also, it shows the liters of water used.



Arduino Cloud project dashboard on a smartphone

tablet, allowing a user to receive updates on the irrigation status from anywhere.

Full Smart Irrigation System Example

All the necessary files to replicate this application note can be found below:

The complete code can be downloaded [here](#).

The Irrigation System Working

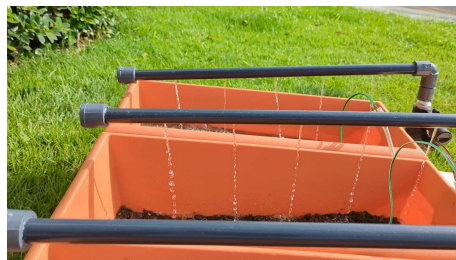
Below you can find some additional images and animations showing how the system works:



Watermark sensors



Battery connection



Remember that the system is designed to be scalable; therefore, it is possible to control 4 bigger irrigation zones

different crops and, consequently,
different water and soil needs.

Conclusion

In this application note, you have learned how to build a LoRa® technology based irrigation system to water your crops automatically or manually and monitor the crop's status remotely. Thanks to the soil moisture analysis, you can avoid irrigation when it's not necessary, saving water and avoiding over-irrigation or flooding problems.

Arduino Edge Control allows you to easily implement this kind of agriculture systems ready for field deployment. Alongside MKR boards, it can get access to the network using the most suitable technology for your application.

In this project, LPWAN was used leveraging its capabilities: this technology is perfect for remote deployments where there is no internet connection and for battery-powered devices because of its low energy consumption.

Thanks to its capabilities of controlling different types of actuators and handling a vast variety of input sensors, the Edge Control is a great choice for developing robust and agriculture environment-proof solutions.

Next Steps

Since you already know how to develop a Smart Irrigation System with Arduino Edge Control and the MKR WAN 1310,

portfolio and integrating it into your professional setup.

We have a similar project using motorized ball valves, WiFi® connectivity and scheduled remote control from the Arduino Cloud, click this [link](#) to know more.

You can extend the capabilities of your Edge Control-based system by adding different connectivity options, leveraging the Arduino MKR family like WiFi®, GSM, RS-485 or Ethernet.

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

